

AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 27.10.2026

Lab 04

Pętle

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć **iterację**: jak powtarzać instrukcje bez kopiowania kodu
- Pętle `for` i `while`, funkcja `range()`, instrukcje `break` i `continue`
- Pętle zagnieżdżone i pierwsze spojrzenie na **koszt algorytmu**
- Wzorce: **akumulator**, **licznik**, **wartownik**
- Efekty z sylabusa: **U1**, **U3** (algorytmy z pętlami), **KS1**

CZĘŚĆ I

Teoria: iteracja

Dwie pętle, dwa pytania

	FOR	WHILE
Pytanie	„dla każdego elementu...”	„ dopóki warunek jest prawdziwy...”
Liczba powtórzeń	znana z góry (długość kolekcji)	nieznana z góry
Typowe użycie	lista, tekst, <code>range()</code>	menu, gra, wczytywanie danych
Ryzyko	małe	pętla nieskończona

```

for letter in "ALK":
    print(letter)           # A, L, K

n = 3
while n > 0:
    print(n)               # 3, 2, 1
    n -= 1

```

`range(start, stop, step)`

WYWOŁANIE	WARTOŚCI
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(1, 6)</code>	1, 2, 3, 4, 5
<code>range(0, 10, 3)</code>	0, 3, 6, 9
<code>range(5, 0, -1)</code>	5, 4, 3, 2, 1

- `stop` **nie należy** do zakresu – tak jak w wycinkach list
- `range` nie tworzy listy w pamięci, tylko **generuje** kolejne liczby: `range(10**12)` zajmuje kilkadziesiąt bajtów

E. W. Dijkstra, „Why numbering should start at zero” (EWD831, 1982) – uzasadnienie przedziałów lewostronnie domkniętych.

Po czym iterujemy?

```
for x in [3, 1, 2]: ...           # elementy listy
for ch in "Python": ...          # znaki tekstu
for key, value in {"a": 1}.items(): ... # pary ze słownika
for i, name in enumerate(["Ala", "Ola"], start=1):
    print(i, name)                # 1 Ala, 2 Ola
for a, b in zip([1, 2], ["x", "y"]):
    print(a, b)                  # 1 x, 2 y
```

- `for` działa z każdym obiektem **iterowalnym** – takim, który potrafi podawać kolejne elementy
- `enumerate` daje numer, `zip` łączy kilka kolekcji „na suwak”

break, continue i else

```
while True:
    line = input("Ocena: ")
    if line == "":
        break          # wyjdź z pętli
    if not line.isdigit():
        continue       # pominięcie reszty tego obrotu
    ...
```

- `break` kończy **najbliższą** pętlę, `continue` przechodzi do kolejnego obrotu
- `else` przy pętli wykonana się, gdy pętla **nie** została przerwana przez `break`:

```
for d in range(2, n):
    if n % d == 0:
        print("złożona"); break
```

```
else:
```

```
    print("pierwsza")
```

Wzorce pętli

WZORZEC	IDEA	PRZYKŁAD
akumulator	zmienna zbiera wynik	<code>total += x</code>
licznik	zlicza wystąpienia	<code>count += 1</code>
wartownik	specjalna wartość kończy dane	pusta linia, <code>-1</code>
szukanie ekstremum	pamiętaj najlepszy dotąd	<code>if x > best: best = x</code>
flaga	pamiętaj, czy coś zaszło	<code>found = True</code>

- Większość algorytmów na tym kursie to kombinacje tych pięciu wzorców

Pętla `while` a poprawność

- Każda pętla `while` potrzebuje **postępu**: czegoś, co zbliża ją do końca

```
# Algorytm Euklidesa (ok. 300 r. p.n.e.) – NWD dwóch liczb
a, b = 1071, 462
while b != 0:
    a, b = b, a % b
print(a)    # 21
```

- Tu postęp jest pewny: `b` maleje przy każdym obrocie, a jest nieujemne
- **Niezmiennik pętli**: własność prawdziwa przed i po każdym obrocie (tu: NWD(a, b) się nie zmienia)
- Zawieszony program przerwiesz w terminalu: **Ctrl+C** (`KeyboardInterrupt`)

Pętle zagnieżdżone i koszt

```
for i in range(1, n + 1):  
    for j in range(1, n + 1):  
        print(f"{i * j:4}", end=" ")  
    print()
```

N	LICZBA MNOŻEŃ
10	100
100	10 000
1000	1 000 000

- Dwie zagnieżdżone pętle po n → n^2 operacji
- Notację Big O poznamy dokładnie w labie 13 (sortowanie i wyszukiwanie)

CZĘŚĆ II

Praktyka

Rozkład liczby na cyfry

```
n = 12345
while n > 0:
    digit = n % 10      # ostatnia cyfra: 5, 4, 3, 2, 1
    n //= 10           # odegnij ostatnią cyfrę
```

N	N % 10	N // 10
12345	5	1234
1234	4	123
...	...	...
1	1	0

- Uwaga na **0**: pętla `while n > 0` nie wykona się ani razu – a 0 ma jedną cyfrę

Ciekawostki

- **Ada Lovelace** w notatkach do maszyny analitycznej Babbage'a (1843, *Note G*) opisała algorytm liczenia liczb Bernoulliego z **powtarzanymi krokami** – uważany za pierwszy program
- 9.09.1947 zespół komputera **Harvard Mark II** wkleił do dziennika ćmę znalezioną w przekaźniku z dopiskiem „*first actual case of bug being found*”
- Siedziba Apple w Cupertino przez lata miała adres **1 Infinite Loop**
- **Problem Collatza** (1937): „jeśli n parzyste $\rightarrow n/2$, inaczej $\rightarrow 3n+1$ ”. Czy pętla zawsze dojdzie do 1? Nikt tego nie udowodnił

Ciekawostki: crawlery

- Wyszukiwarki indeksują internet **crawlerami** – programami, które w pętli pobierają strony i dodają znalezione linki do kolejki
- Uprzejmy crawler czyta najpierw plik **robots.txt** – konwencja z 1994 r. (Martijn Koster), od 2022 r. standard **RFC 9309**
- Nasz mini-crawler czyta lokalny plik, więc nikomu nie przeszkadza

Dzisiejsze zadania

PLIK	ZADANIE
★ <code>zad1_srednia.py</code>	średnia ocen do pustej linii
★ <code>zad2_tabliczka.py</code>	tabliczka mnożenia $n \times n$
★ <code>zad3_suma_cyfr.py</code>	suma i liczba cyfr
★ <code>zad4_zgadywanka.py</code>	zgadnij liczbę
☆ <code>zad5_crawler.py</code>	mini-crawler

Sprawdzenie i oddanie

```
cd lab04_petle
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 04"
git push
```

- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 11:45**
- Po zajęciach – **0 pkt**, bez wyjątków
- Pamiętaj: **Sync fork** na początku zajęć, żeby pobrać nowy lab

Do poczytania

- E. Matthes, *Python. Instrukcje dla programisty*, Helion 2023 – rozdz. 4 i 7
- M. Dawson, *Python dla każdego*, Helion 2015 – rozdz. 3–4
- Dokumentacja: *More Control Flow Tools* (docs.python.org/3/tutorial/controlflow.html)

Następny lab (03.11): listy i krotki